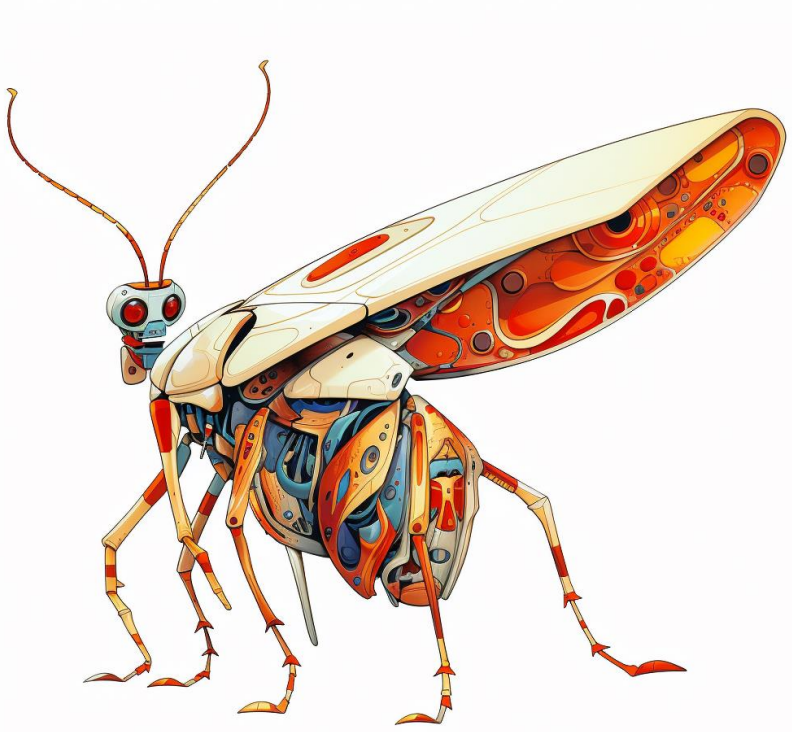
Debugging

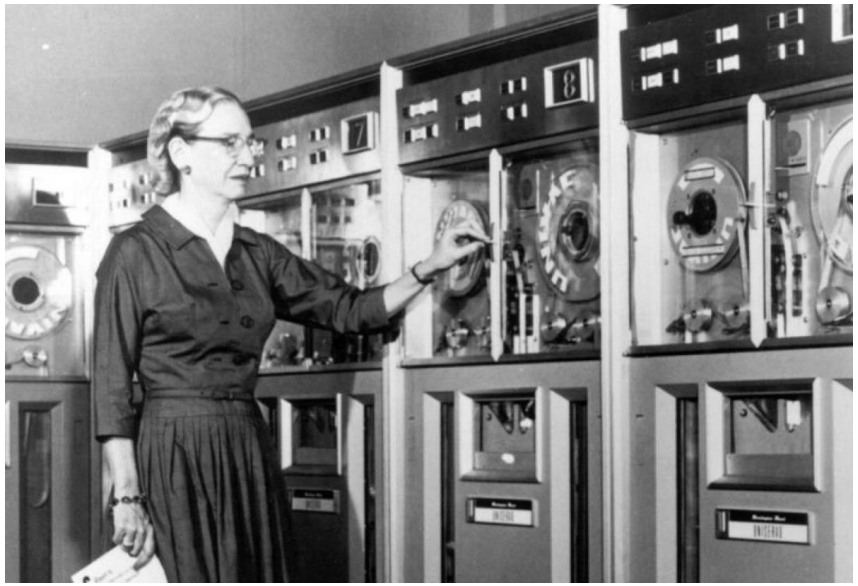
How to fix problems

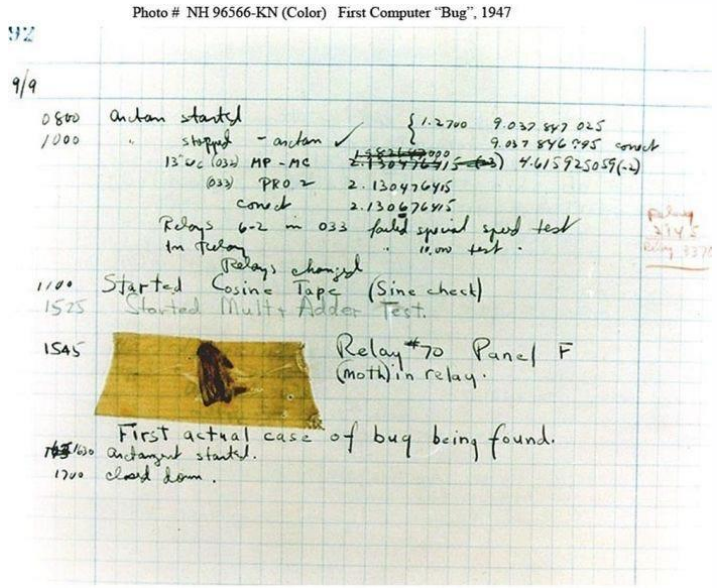


Lisa, Carlos, and Pietro

The very first computer bug

Team around Grace Hopper at Harvard found a moth in their computer in 1947 in what is maybe the first description of a computer bug

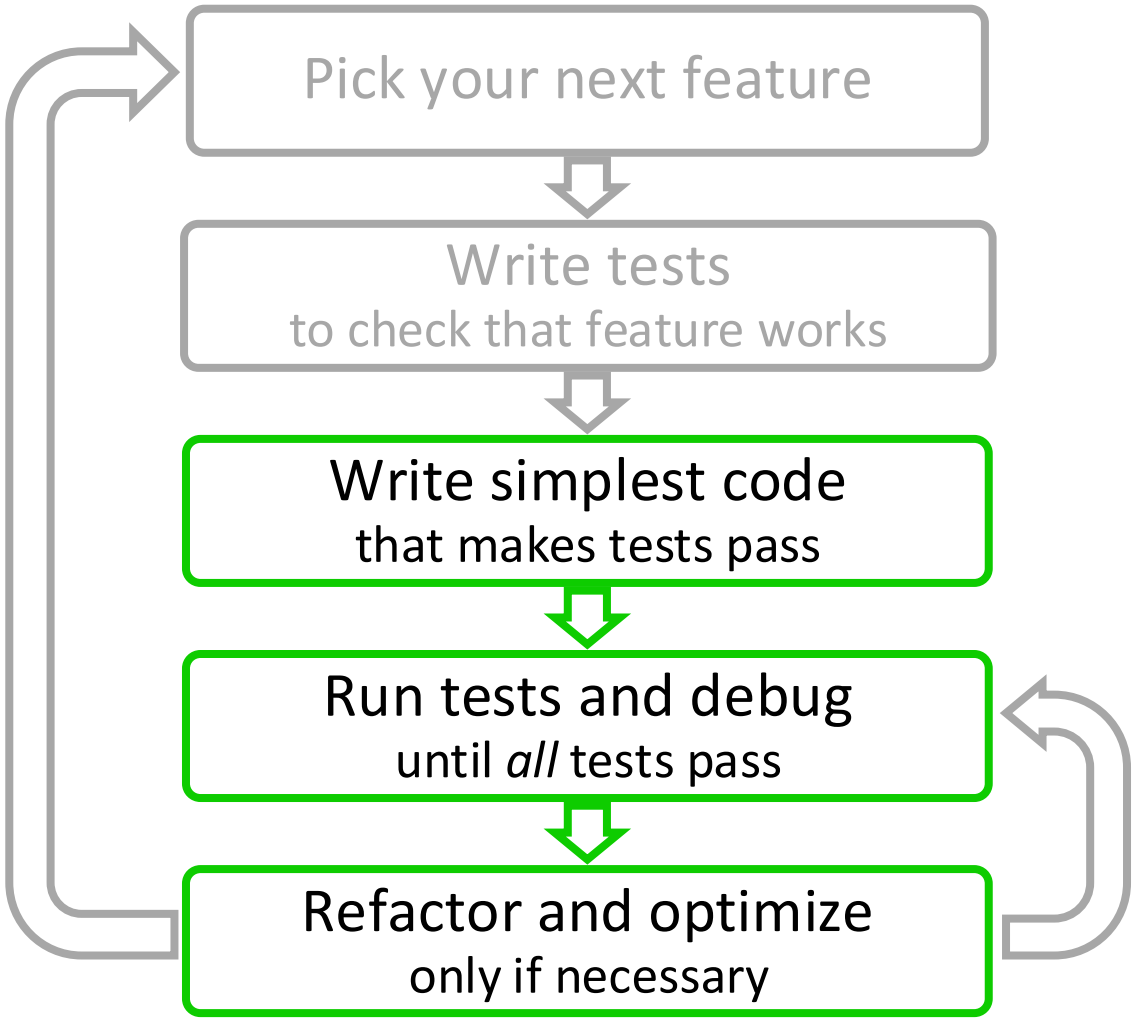




Debugging and the agile development cycle

- By writing tests, you *anticipate* the bugs
- Your test cases should already exclude a big portion of the possible causes

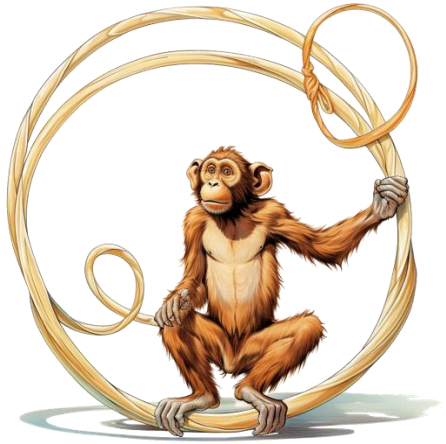
Debugging and the agile development cycle



`pytest`

pdb

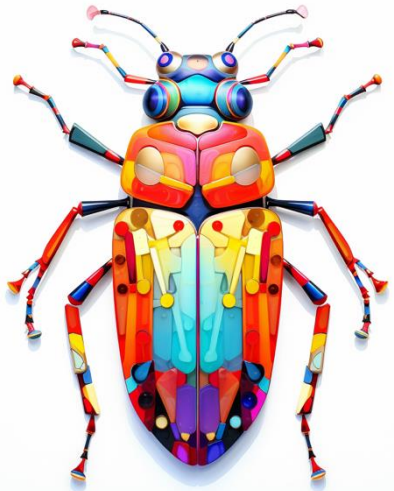
`timeit`
`cProfile`
`line_profiler`



Debugging and the agile development cycle

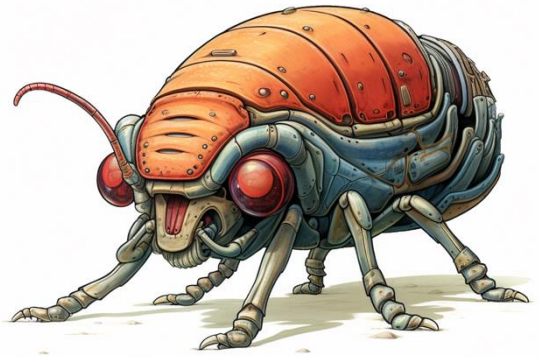
1. Add a test that matches the behavior you expect. It will fail and reproduce the bug
2. Debug and fix the the bug
3. Run the tests until they all pass (go back to 2 if necessary)

- **Now your bug is fixed *and* it will never occur again!**



pdb, the Python debugger

- Core idea in debugging: you can stop the execution of your application at any point, look at the state of the variables, and execute the code step by step
- Command-line based debugger
- pdb opens an interactive shell, in which one can interact with the code
  - examine and change value of variables
  - execute code line by line
  - set up breakpoints
  - examine calls stack



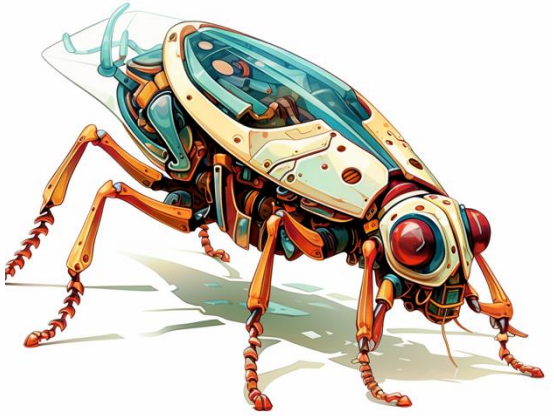
Entering the debugger

- Enter at a specific point in the code (easy alternative to `print`):

```
# some code here
# the debugger starts here
breakpoint()
# rest of the code
```

Demonstration!

- Debugging with pdb



Hands-on! – Issue #2

1) Go to `local_maxima_part3_debug/`**`local_maxima.py`** and execute it

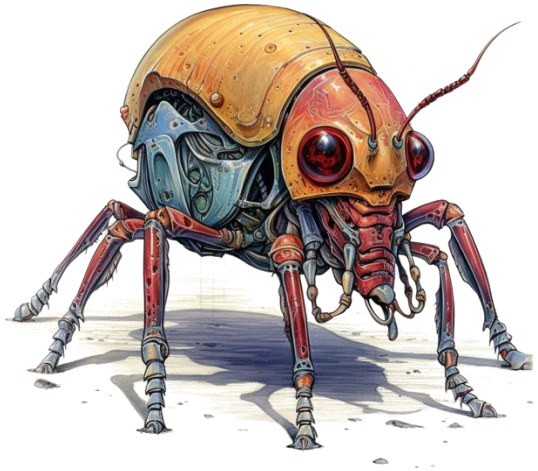
2) Run the script with these inputs:

   - [1] (expected output: [0]),
   - [2, 2, 1, 0] (expected output: [0])

3) When you find a bug, write a test, then debug and fix it. Finally, create a PR

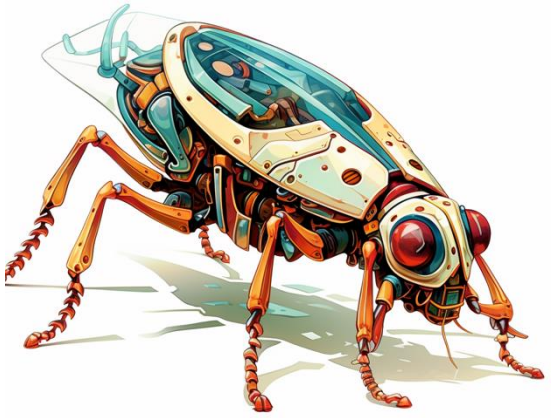
pdb cheatsheet:

h (help) [*command*]	print help about *command*
n (next)	execute current line of code, go to next line
c (continue)	continue executing the program until next breakpoint, exception, or end of the program
s (step into)	execute current line of code; if a function is called, follow execution inside the function
l (list)	print code around the current line
w (where)	show a trace of the function call that led to the current line
p (print)	print the value of a variable
q (quit)	leave the debugger
b (break) [*lineno* | *function*[, *condition*]]	set a breakpoint at a given line number or function, stop execution there if *condition* is fulfilled
cl (clear)	clear a breakpoint
! (execute)	execute a python command
<enter>	repeat last command

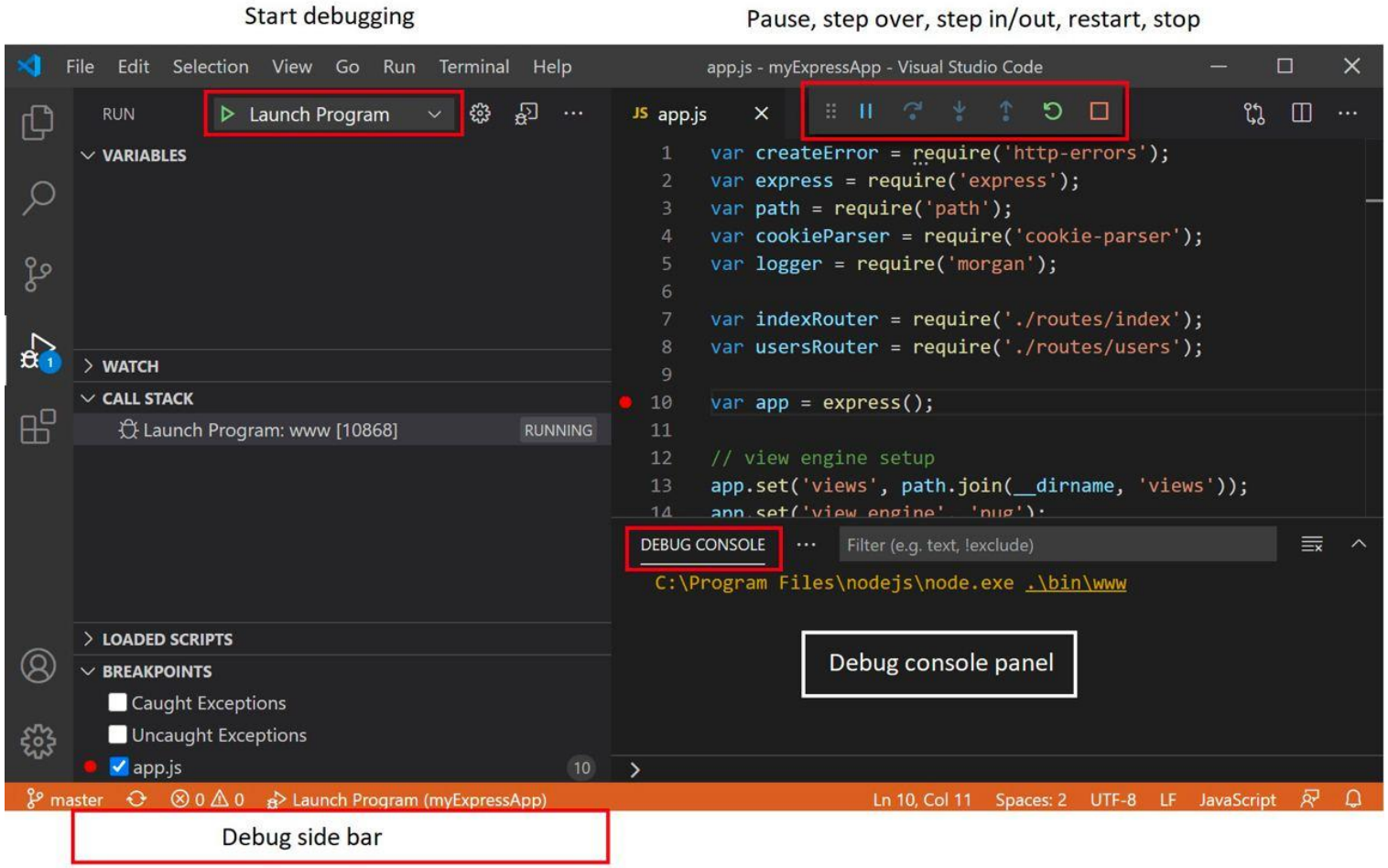


Demonstration!

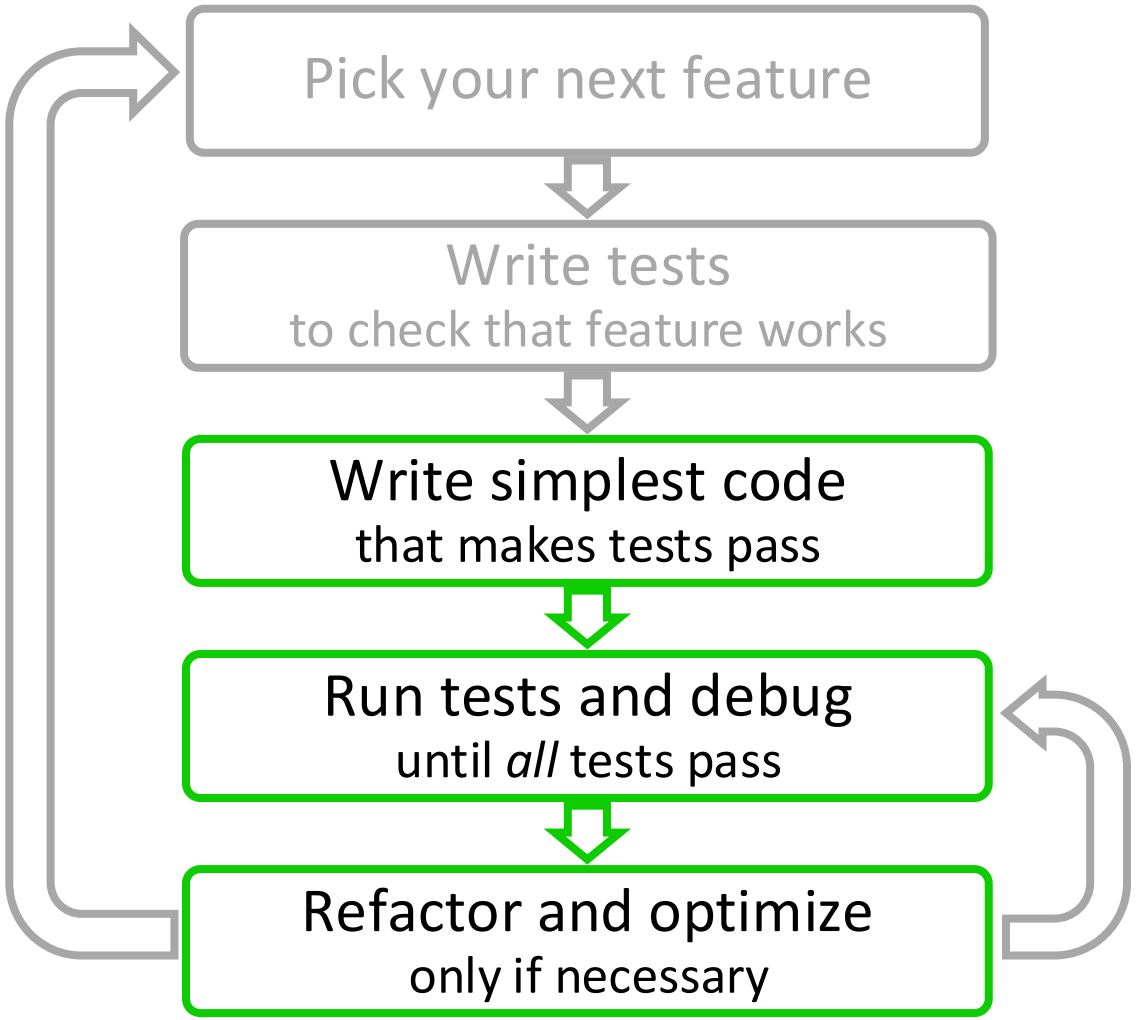
- Debugging with pdb
- Debugging from an IDE



Entering the debugger from VSCode



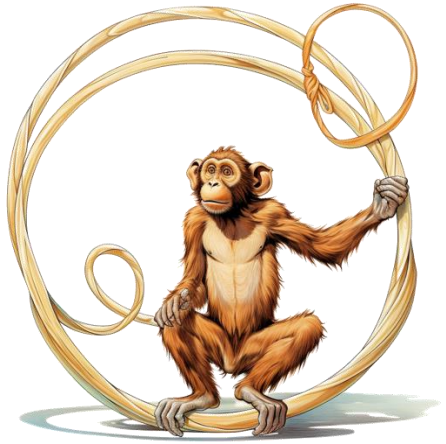
Test, debug, then commit and create a PR



`pytest`

pdb

`timeit`
`cProfile`
`line_profiler`



The End

Entering the debugger from Jupyter

- %pdb – preventive
- %debug – post-mortem

Hands-on!

- Go to `bug_hunt/file_datastore.py` and execute it

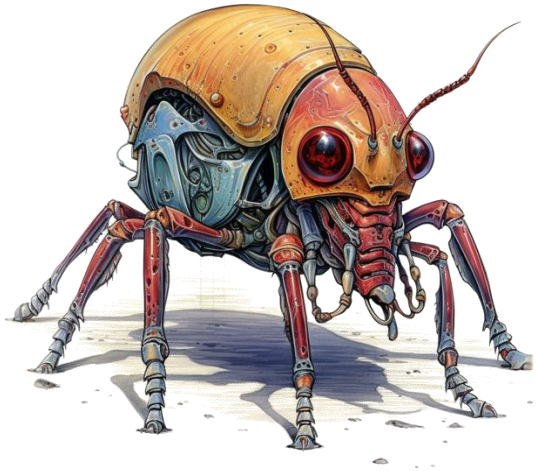
```
data = b'A test! 012'
datastore = FileDatastore(base_path='./datastore')
datastore.write('a/mydata.bin', data)

# This should pass!
assert os.path.exists('./datastore/a/mydata.bin’)
```

- It fails! But… it works when the base_path is an absolute path :-(

Change this exercise!!!!

Maybe debug a script or maybe even better - a test?



Static checking and linting

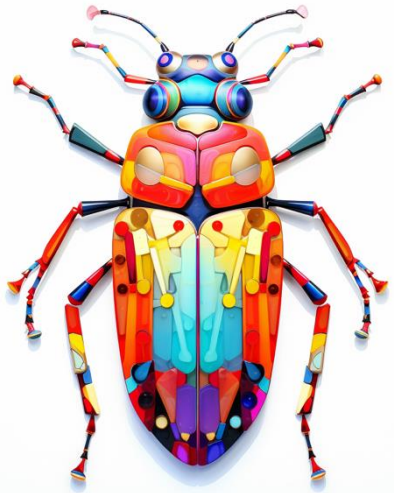
One of the problems with debugging in Python is that mo
appear when the code executes.

"Static checking" tools analyze the code without executin

- `pep8`: check that the style of the files is compatible wit
- `pyflakes`: look for errors like defined but unused variab
  undefined names, etc.
- `flake8`: pep8 and pyflakes in a single, handy command

- and also: `yapf, black, …`

Modify this and adapt to not have overlap with organising lecture



Hands-on!

- Run `flake8` on one the files you edited today